

Documentación de la Práctica de Laboratorio

Visualización en Tiempo Real de Temperatura y Humedad con Raspberry Pi y OLED SSD1306

1. Introducción

En esta práctica se desarrolla un sistema capaz de medir la temperatura y humedad del ambiente utilizando un sensor DHT11 conectado a una Raspberry Pi. Los datos obtenidos se muestran en tiempo real en una pantalla OLED SSD1306 mediante texto, íconos y gráficas animadas.

El proyecto permite aprender sobre:

- Sensores ambientales.
 - Comunicación I2C.
 - Programación en Python.
 - Visualización gráfica en pantallas OLED.
 - Programación modular y orientada a objetos.
-

2. Objetivo General

Diseñar e implementar un sistema de monitoreo ambiental utilizando Raspberry Pi, un sensor DHT11 y una pantalla OLED SSD1306 para visualizar temperatura y humedad en tiempo real.

3. Objetivos Específicos

- Leer datos del sensor DHT11.
- Mostrar información en una pantalla OLED.
- Crear gráficas en tiempo real.
- Aplicar programación modular.

- Implementar manejo de errores.
- Utilizar íconos gráficos para mejorar la interfaz.

4. Materiales Utilizados

Cantidad Material

1	Raspberry Pi
1	Sensor DHT11
1	Pantalla OLED SSD1306
1	Protoboard
Varios	Jumpers
2	Íconos BMP
1	Fuente de alimentación

5. Librerías Utilizadas

Librería	Función
adafruit_dht	Lectura del sensor DHT11
adafruit_ssd1306	Control de la pantalla OLED
Pillow	Manejo de imágenes y gráficos

6. Instalación de Dependencias

Actualizar el sistema:

```
sudo apt update
```

Instalar pip:

```
sudo apt install python3-pip
```

Instalar librerías:

```
pip3 install adafruit-circuitpython-dht
```

```
pip3 install adafruit-circuitpython-ssd1306
```

```
pip3 install pillow
```

7. Conexiones del Circuito

Conexión del Sensor DHT11

DHT11 Raspberry Pi

VCC 3.3V

DATA GPIO17

GND GND

Conexión de la Pantalla OLED

OLED SSD1306 Raspberry Pi

VCC 3.3V

GND GND

SCL GPIO3

SDA GPIO2

8. Estructura del Proyecto

```
sensor_display_project/
```

```
|
```

```
├— main.py
```

```
├— config.py
```

```
├── sensor_dht.py
├── oled_visualizer.py
├── utils.py
├── icons/
│   ├── temp.bmp
│   └── hum.bmp
└── README.md
```

9. Funcionamiento del Sistema

El sistema funciona de la siguiente manera:

1. El sensor DHT11 mide temperatura y humedad.
 2. La Raspberry Pi procesa los datos.
 3. La pantalla OLED muestra:
 - Temperatura.
 - Humedad.
 - Gráficas en tiempo real.
 - Íconos animados.
 4. Los datos se actualizan cada 2 segundos.
-

10. Explicación de los Archivos

10.1 config.py

Este archivo contiene todas las configuraciones generales del sistema.

Configura:

- Pines GPIO.
- Tamaño de pantalla.

- Límites de temperatura y humedad.
- Tiempo de actualización.
- Rutas de los íconos.

Código:

```
import board
```

```
I2C_SCL = board.SCL
I2C_SDA = board.SDA
DHT_PIN = board.D17
```

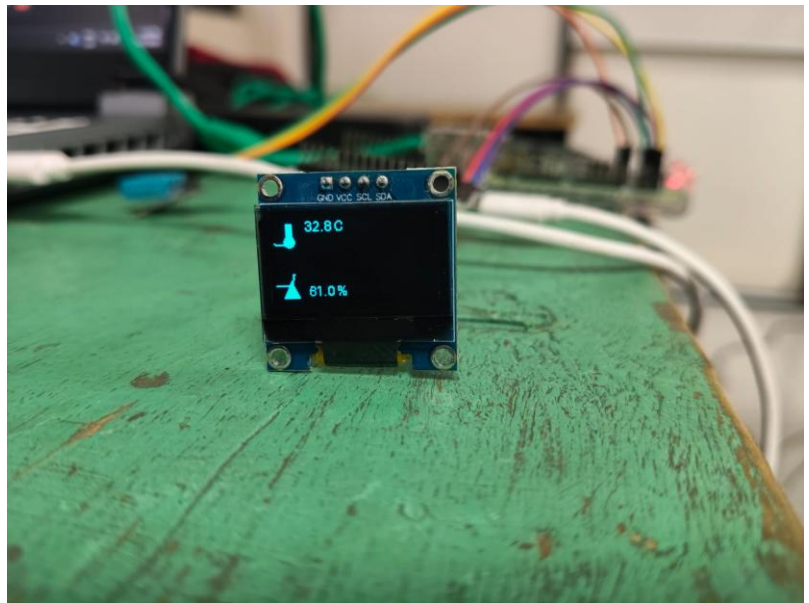
```
OLED_WIDTH = 128
OLED_HEIGHT = 64
```

```
TEMP_MAX = 50
HUM_MAX = 100
```

```
HIST_MAX_POINTS = 128
REFRESH_DELAY = 2
```

```
ANIM_LIMIT = 2
```

```
ICON_TEMP_PATH = "icons/temp.bmp"
ICON_HUM_PATH = "icons/hum.bmp"
```



10.2 utils.py

Contiene funciones auxiliares.

Función escalar()

Convierte los valores del sensor en coordenadas para dibujar las gráficas.

Código:

```
def escalar(valor, valor_maximo, alto):  
    return int((1 - valor / valor_maximo) * alto)
```

10.3 sensor_dht.py

Se encarga de leer la información del sensor DHT11.

Funciones principales:

- Leer temperatura.
- Leer humedad.
- Detectar errores.

Código:

```
import adafruit_dht  
from config import DHT_PIN
```

```
class SensorDHT:
```

```
    def __init__(self, pin=DHT_PIN):  
        self.dht = adafruit_dht.DHT11(pin)
```

```
    def leer(self):
```

```
        try:
```

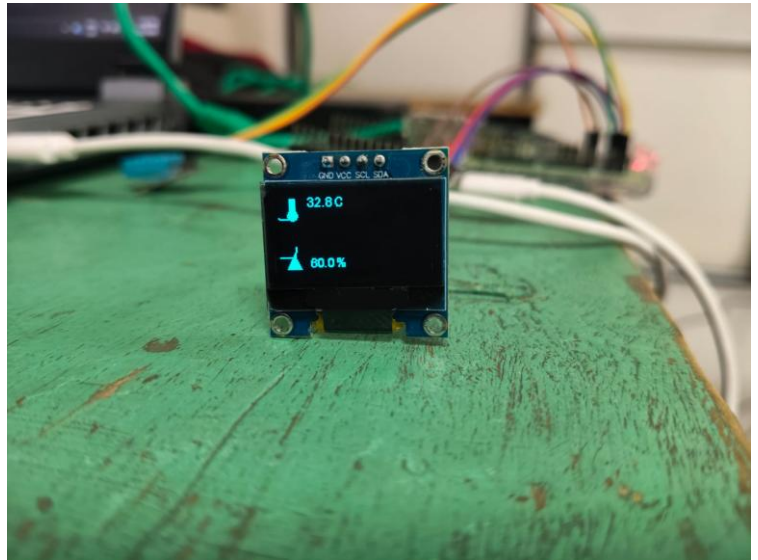
```
            temp = self.dht.temperature  
            hum = self.dht.humidity
```

```
            if temp is not None and hum is not None:  
                return temp, hum
```

```
        else:
```

```
            raise ValueError("Lectura inválida")
```

```
    except RuntimeError as e:
```



```
print("Error:", e.args[0])  
return None, None
```

```
def cerrar(self):  
    self.dht.exit()
```

10.4 oled_visualizer.py

Controla la pantalla OLED.

Funciones:

- Mostrar texto.
- Dibujar gráficas.
- Mostrar íconos.
- Crear animaciones.

Código resumido:

```
self.image = Image.new("1", (width, height))  
self.draw = ImageDraw.Draw(self.image)
```

```
self.draw.text((20, 0), f"{temp:.1f} C")
```

```
self.display.image(self.image)  
self.display.show()
```

10.5 main.py

Archivo principal del sistema.

Funciones:

- Inicializar dispositivos.
- Leer datos continuamente.

- Actualizar la pantalla.
- Guardar historial.

Código principal:

while True:

```
temp, hum = sensor.leer()
```

```
if temp is not None:
```

```
temp_hist.append(temp)
```

```
hum_hist.append(hum)
```

```
visual.mostrar_datos(
```

```
temp,
```

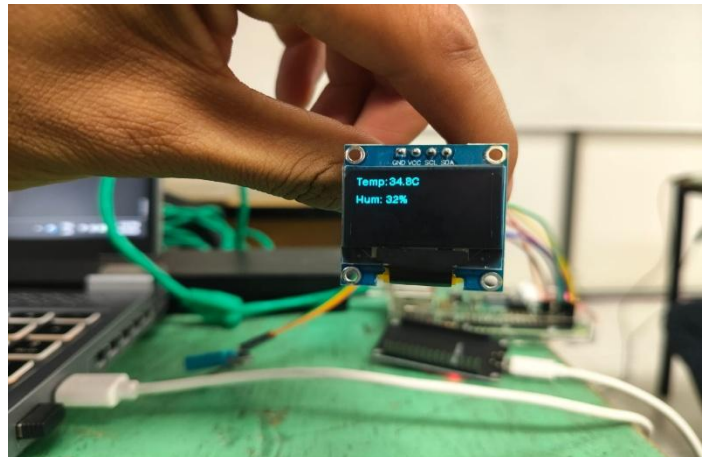
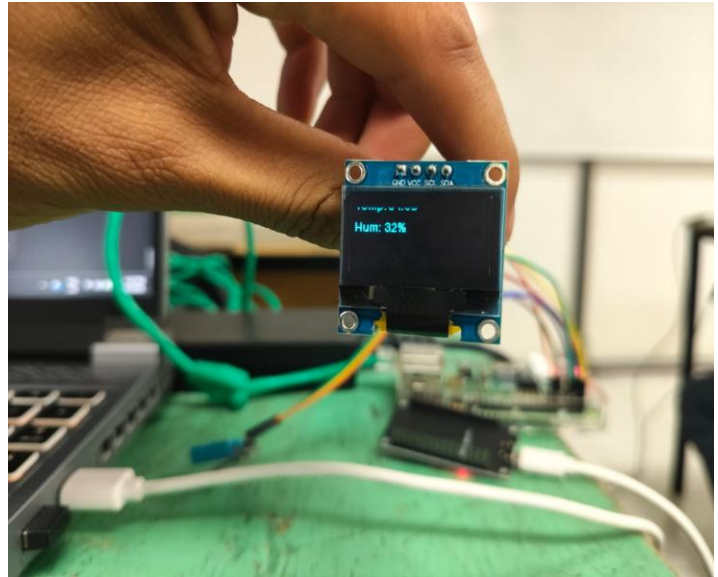
```
hum,
```

```
temp_hist,
```

```
hum_hist
```

```
)
```

```
time.sleep(REFRESH_DELAY)
```



11. Explicación de las Gráficas

Las gráficas muestran el historial de temperatura y humedad.

Funcionamiento:

- Cada nuevo dato se guarda en una lista.
- Los valores se convierten en coordenadas.
- Se dibujan líneas entre puntos consecutivos.

Esto permite visualizar cambios en tiempo real.

12. Animación de Íconos

Los íconos se mueven ligeramente hacia arriba y abajo usando la variable:

`anim_offset`

Esto hace que la interfaz sea más dinámica y atractiva.

13. Manejo de Errores

El sistema utiliza excepciones para evitar que el programa se detenga si el sensor falla.

Ejemplo:

```
except RuntimeError:  
    return None, None
```

Esto mejora la estabilidad del proyecto.

14. Ejecución del Proyecto

Entrar a la carpeta:

```
cd sensor_display_project
```

Ejecutar:

```
python3 main.py
```

15. Actividades Realizadas

- Conexión del DHT11.
- Configuración del bus I2C.
- Visualización de datos.
- Creación de gráficas.
- Implementación de animaciones.

- Manejo de errores.

16. Mejoras Propuestas

- Guardar datos en CSV.
- Enviar información a internet.
- Agregar alertas visuales.
- Crear una interfaz web.
- Implementar modo nocturno.

17. Conclusión

Se desarrolló un sistema de monitoreo ambiental funcional utilizando Raspberry Pi y sensores electrónicos. El proyecto permitió integrar hardware y software mediante Python, aplicando programación modular y visualización gráfica en tiempo real. Además, se comprendió el funcionamiento de sensores ambientales, pantallas OLED y manejo de datos gráficos dentro de sistemas embebidos.

